

Ứng dụng chat E2EE - Implement

Sinh viên thực hiện: Lê Thị Hồng Hạnh - 22127103

1. Tables

Các bảng ghi dữ liệu được lưu trữ ở client browser (IndexedDB)

Identity: Cặp khóa định danh người dùng

Thuộc tính	Kiểu dữ liệu	Khóa	Mô tả
publicKey	String	Primary key	Khóa công khai cũng là định danh duy nhất
privateKey	String		Khóa bí mật

Channel: Một cuộc hội thoại

Thuộc tính	Kiểu dữ liệu	Khóa	Mô tả
channelId	String	Primary key	ID cuộc trò chuyện
peerPublicKey	String		Khóa công khai của đối phương
sharedKey	String		Khóa bí mật chung mã hóa tin nhắn
createdAt	Date		Thời điểm tạo cuộc hội thoại
nickname	String		Tên cuộc trò chuyện thay vì mặc định là channel ID

Message: Một tin nhắn

Thuộc tính	Kiểu dữ liệu	Khóa	Mô tả
messageId	String	Primary key	ID tin nhắn
channelId	String	Foreign key	Tin nhắn thuộc phòng chat nào
sender	String		'me' hoặc 'peer'
content	String		Nội dung tin nhắn
timestamp	Date		Thời gian gửi/nhận
status	String		Trạng thái tin nhắn (đã gửi, đã nhận, gửi thất bại,...)

2. Functions/Procedures

2.1. Client-side

a. Database functions

1. Hàm `getDB`

- **Mô tả:** Khởi tạo và trả về instance của cơ sở dữ liệu IndexedDB dưới dạng Singleton, đồng thời quản lý việc tạo bảng/phiên bản cơ sở dữ liệu.
- **Input:** Không có.
- **Output:** Instance kết nối với IndexedDB.
- **Thuật toán/Các bước thực hiện:**
 - Kiểm tra nếu biến `_db` (instance đã khởi tạo) đã tồn tại thì trả về ngay.
 - Nếu chưa, gọi hàm `openDB` để khởi tạo database có tên `'cipher-chat'` ở phiên bản 2.
 - Thực hiện kịch bản `upgrade` (chạy khi database chưa tồn tại): Tạo các object store `identity`, `channels`, và `messages` (với index `by-channel`).
 - Gán kết quả vào `_db` và trả về.

2. Hàm `saveIdentity`

- **Mô tả:** Lưu trữ thông tin khóa định danh của người dùng (Identity) vào cơ sở dữ liệu.
- **Input:**
 - `publicKeyRaw` (string): Khóa công khai dạng thô.
 - `privateKeyJwk` (string): Khóa bí mật dạng JWK.
- **Output:** Thông tin định danh người dùng được lưu vào IndexedDB.
- **Thuật toán/Các bước thực hiện:**
 - Lấy instance database qua `getDB()`.
 - Lưu hoặc ghi đè record vào store `identity` với key cố định là `'singleton'`.

3. Hàm `loadIdentity`

- **Mô tả:** Lấy thông tin khóa định danh của người dùng hiện tại từ cơ sở dữ liệu.
- **Input:** Không có.
- **Output:** Thông tin định danh người dùng.
- **Thuật toán/Các bước thực hiện:**
 - Lấy instance database qua `getDB()`.
 - Truy vấn và trả về dữ liệu từ store `identity` thông qua khóa `'singleton'`.

4. Hàm `saveChannel`

- **Mô tả:** Lưu thông tin của một kênh trò chuyện (channel) vào cơ sở dữ liệu.
- **Input:** `record` (ChannelRecord): Đối tượng chứa thông tin kênh (ID, khóa chia sẻ, ngày tạo, nickname...).
- **Output:** Thông tin kênh trò chuyện được lưu vào IndexedDB
- **Thuật toán/Các bước thực hiện:**
 - Lấy instance database qua `getDB()`.
 - Đẩy record mới vào store `channels`.

5. Hàm `loadChannel`

- **Mô tả:** Lấy thông tin chi tiết của một kênh trò chuyện dựa trên ID, nếu người dùng có đặt `nickname` cho cuộc trò chuyện gán cho trường `nickname`.
- **Input:** `channelId` (string): ID của kênh cần lấy.
- **Output:** Thông tin cuộc trò chuyện.
- **Thuật toán/Các bước thực hiện:**
 - Lấy database qua `getDB()`.
 - Tìm record trong store `channels` qua `channelId`.
 - Trả về record.

6. Hàm `loadAllChannels`

- **Mô tả:** Lấy danh sách tất cả các kênh trò chuyện hiện có và sắp xếp theo thời gian tạo mới nhất.
- **Input:** Không có.
- **Output:** Danh sách tất cả cuộc trò chuyện.
- **Thuật toán/Các bước thực hiện:**
 - Lấy database qua `getDB()`.
 - Lấy toàn bộ record từ store `channels`.
 - Sắp xếp danh sách giảm dần theo trường `createdAt` và trả về kết quả.

7. Hàm `updateChannelNickname`

- **Mô tả:** Chỉ cập nhật lại tên hiển thị (nickname) cho một kênh trò chuyện đã tồn tại.
- **Input:**

- `channelId` (string): ID của kênh cần đổi tên.
- `nickname` (string): Tên hiển thị mới.
- **Output:** Nickname của cuộc trò chuyện được cập nhật và lưu vào IndexedDB.
- **Thuật toán/Các bước thực hiện:**
 - Lấy database qua `getDB()`.
 - Truy xuất kênh theo `channelId`.
 - Nếu kênh tồn tại, sao chép record hiện tại, ghi đè trường `nickname` bằng giá trị mới và lưu lại vào store `channels`.

8. Hàm `deleteChannel`

- **Mô tả:** Xóa vĩnh viễn một kênh trò chuyện cùng với toàn bộ tin nhắn thuộc kênh đó.
- **Input:** `channelId` (string): ID của kênh cần xóa.
- **Output:** Thông tin cuộc trò chuyện được xóa khỏi IndexedDB.
- **Thuật toán/Các bước thực hiện:**
 - Lấy database qua `getDB()`.
 - Lấy danh sách các khóa (key) của tất cả tin nhắn thuộc kênh này từ index `'by-channel'`.
 - Mở một transaction ở chế độ `readwrite` tác động lên cả store `channels` và `messages`.
 - Dùng vòng lặp xóa từng tin nhắn trong store `messages` qua các key vừa lấy.
 - Xóa kênh trong store `channels` theo `channelId`.

9. Hàm `saveMessage`

- **Mô tả:** Lưu một tin nhắn mới vào cơ sở dữ liệu.
- **Input:** `msg` (Message): Đối tượng tin nhắn cần lưu.
- **Output:** Tin nhắn được lưu vào IndexedDB.
- **Thuật toán/Các bước thực hiện:**
 - Lấy database qua `getDB()`.
 - Lưu thông tin tin nhắn vào store `messages`.

10. Hàm `loadMessages`

- **Mô tả:** Tải toàn bộ danh sách tin nhắn của một kênh cụ thể, sắp xếp theo thứ tự thời gian (từ cũ đến mới).

- **Input:** `channelId` (string): ID của kênh trò chuyện.
- **Output:** Danh sách tin nhắn của cuộc trò chuyện.
- **Thuật toán/Các bước thực hiện:**
 - Lấy database qua `getDB()`.
 - Dùng index `'by-channel'` để lấy tất cả tin nhắn thuộc `channelId` từ store `messages`.
 - Ép kiểu (type-cast) trường `status` cho hợp lệ với TypeScript.
 - Sắp xếp mảng tăng dần theo trường `timestamp` và trả về.

11. Hàm `updateMessageStatus`

- **Mô tả:** Cập nhật trạng thái của một tin nhắn (ví dụ: đang gửi, đã gửi, lỗi).
- **Input:**
 - `id` (string): ID của tin nhắn.
 - `status` (`Message['status']`): Trạng thái mới của tin nhắn.
- **Output:** Trạng thái của tin nhắn được cập nhật.
- **Thuật toán/Các bước thực hiện:**
 - Lấy database qua `getDB()`.
 - Tìm tin nhắn trong store `messages` theo `id`.
 - Nếu tồn tại, cập nhật thuộc tính `status` và lưu đè lại vào database.

12. Hàm `loadLastMessage`

- **Mô tả:** Lấy tin nhắn mới nhất của một kênh trò chuyện.
- **Input:** `channelId` (string): ID của kênh trò chuyện.
- **Output:** Tin nhắn mới nhất hoặc `null`
- **Thuật toán/Các bước thực hiện:**
 - Lấy database qua `getDB()`.
 - Lấy tất cả tin nhắn của kênh qua index `'by-channel'`.
 - Kiểm tra nếu mảng rỗng thì trả về `undefined`.
 - Sắp xếp mảng giảm dần theo thời gian (`timestamp`) để đưa tin nhắn mới nhất lên đầu.
 - Trả về phần tử đầu tiên.

b. Cryptographic functions

1. Hàm `generateIdentityKeyPair`

- **Mô tả:** Tạo một cặp khóa cho định danh người dùng sử dụng thuật toán ECDH với đường cong P-256.
- **Input:** Không có.
- **Output:** Cặp khóa mã hóa theo chuẩn Web Crypto API.
- **Thuật toán/Các bước thực hiện:**
 - Gọi API `crypto.subtle.generateKey` với cấu hình thuật toán `ECDH` (P-256).
 - Thiết lập khóa có thể trích xuất (`extractable = true`) và được phép dùng cho mục đích tạo khóa dẫn xuất (`deriveKey`).

2. Hàm `exportPublicKey`

- **Mô tả:** Xuất khóa công khai từ dạng đối tượng (`CryptoKey`) sang dạng chuỗi Base64.
- **Input:** `key` (`CryptoKey`): Khóa công khai cần xuất.
- **Output:** Khóa công khai dạng chuỗi Base64.
- **Thuật toán/Các bước thực hiện:**
 - Gọi `crypto.subtle.exportKey` với định dạng `raw` để lấy dữ liệu thô (`ArrayBuffer`).
 - Chuyển đổi dữ liệu thô thành chuỗi Base64 bằng hàm helper `bufferToBase64`.

3. Hàm `importPublicKey`

- **Mô tả:** Nhập dữ liệu khóa công khai từ định dạng Base64 thành đối tượng `CryptoKey`.
- **Input:** `base64` (string): Chuỗi Base64 chứa dữ liệu khóa công khai.
- **Output:** Khóa công khai dạng `CryptoKey`
- **Thuật toán/Các bước thực hiện:**
 - Giải mã chuỗi Base64 thành mảng byte (`Uint8Array`) bằng hàm `base64ToBuffer`.
 - Gọi `crypto.subtle.importKey` nhập mảng byte dạng `raw` thành `CryptoKey` dùng thuật toán ECDH.

4. Hàm `exportPrivateKey`

- **Mô tả:** Xuất khóa bí mật từ dạng đối tượng sang chuỗi JSON theo chuẩn JWK (JSON Web Key).
- **Input:** `key` (`CryptoKey`): Khóa bí mật cần xuất.
- **Output:** Khóa bí mật dạng chuỗi JSON JWK.

- **Thuật toán/Các bước thực hiện:**

- Gọi `crypto.subtle.exportKey` với định dạng `jwk` để lấy đối tượng JWK.
- Chuyển đối tượng JWK thành chuỗi chuỗi JSON thông qua `JSON.stringify`.

5. Hàm `importPrivateKey`

- **Mô tả:** Nhập khóa bí mật từ chuỗi JSON định dạng JWK thành đối tượng `CryptoKey`.
- **Input:** `json` (string): Chuỗi JSON chứa thông tin JWK của khóa bí mật.
- **Output:** Khóa bí mật dạng `CryptoKey`
- **Thuật toán/Các bước thực hiện:**
 - Phân tích chuỗi JSON (`JSON.parse`) thành đối tượng JWK.
 - Gọi `crypto.subtle.importKey` nhập đối tượng JWK thành `CryptoKey` dùng thuật toán ECDH, mục đích là `deriveKey`.

6. Hàm `deriveSharedKey`

- **Mô tả:** Sinh ra khóa đối xứng dùng chung (shared secret) dựa trên khóa bí mật của người dùng và khóa công khai của người nhận.
- **Input:**
 - `privateKey` (`CryptoKey`): Khóa bí mật của bản thân.
 - `peerPublicKey` (`CryptoKey`): Khóa công khai của đối tác.
- **Output:** Khóa đối xứng AES-GCM 256-bit.
- **Thuật toán/Các bước thực hiện:**
 - Gọi `crypto.subtle.deriveKey` kết hợp thuật toán ECDH trên hai khóa đầu vào.
 - Đầu ra là một khóa AES-GCM có độ dài 256 bit.
 - Cấu hình khóa sinh ra không được phép trích xuất khỏi bộ nhớ (`extractable: false`) và chỉ cấp quyền `encrypt` / `decrypt`.

7. Hàm `encryptMessage`

- **Mô tả:** Mã hóa văn bản thành bản mã (ciphertext) sử dụng khóa chia sẻ chung AES-GCM.
- **Input:**
 - `sharedKey` (`CryptoKey`): Khóa đối xứng AES-GCM dùng chung.
 - `plaintext` (string): Nội dung tin nhắn dạng văn bản.
- **Output:** Ciphertext và IV dạng chuỗi Base64.

- **Thuật toán/Các bước thực hiện:**

- Tạo ngẫu nhiên một Vector khởi tạo (IV) dài 12 bytes (96-bit).
- Mã hóa chuỗi đầu vào (`plaintext`) thành mảng byte thông qua `TextEncoder` .
- Tiến hành mã hóa AES-GCM trên mảng byte bằng `crypto.subtle.encrypt` .
- Chuyển đổi ciphertext và IV sang định dạng Base64 để trả về.

8. Hàm `decryptMessage`

- **Mô tả:** Giải mã bản mã Base64 về văn bản gốc dựa trên khóa chia sẻ chung và Vector khởi tạo (IV).

- **Input:** *

- `sharedKey` (CryptoKey): Khóa đối xứng AES-GCM dùng chung.
- `ciphertext` (string): Bản mã dạng Base64.
- `iv` (string): Vector khởi tạo dạng Base64.

- **Output:** Dữ liệu sau khi giải mã.

- **Thuật toán/Các bước thực hiện:**

- Đưa `ciphertext` và `iv` từ Base64 về mảng byte qua `base64ToBuffer` .
- Thực hiện giải mã bằng `crypto.subtle.decrypt` theo chuẩn AES-GCM.
- Chuyển mảng byte được giải mã thành chuỗi văn bản thông qua `TextDecoder` .

9. Hàm `keyFingerprint`

- **Mô tả:** Tạo ra một chuỗi nhận dạng (fingerprint) ngắn gọn và dễ đọc từ khóa công khai, dùng để xác minh người dùng bằng mắt thường.

- **Input:** `publicKeyRaw` (string): Khóa công khai định dạng Base64.

- **Output:** Chuỗi fingerprint in hoa chứa 16 ký tự hex, gom nhóm 4.

- **Thuật toán/Các bước thực hiện:**

- Đổi khóa công khai Base64 về mảng byte.
- Băm mảng byte này bằng thuật toán SHA-256 (`crypto.subtle.digest`).
- Lấy 8 byte đầu tiên (16 ký tự hex) của mảng băm.
- Định dạng chuỗi hex thành các cụm 4 ký tự (ví dụ: `XXXX XXXX XXXX XXXX`), viết hoa và trả về.

10. Hàm `bufferToBase64`

- **Mô tả:** Hàm phụ trợ dùng để chuyển đổi dữ liệu dạng nhị phân (Buffer) sang chuỗi Base64.
- **Input:** `buffer` (ArrayBuffer | Uint8Array): Khối dữ liệu nhị phân.
- **Output:** Chuỗi Base64 (string).
- **Thuật toán/Các bước thực hiện:**
 - Đảm bảo input là đối tượng `Uint8Array`.
 - Lặp qua từng byte, chuyển thành ký tự tương ứng để nối lại thành chuỗi nhị phân (binary string).
 - Encode chuỗi này thành Base64 thông qua hàm `btoa`.

11. Hàm `base64ToBuffer`

- **Mô tả:** Hàm phụ trợ dùng để chuyển đổi chuỗi Base64 ngược về dạng dữ liệu nhị phân (Uint8Array).
- **Input:** `base64` (string): Chuỗi định dạng Base64.
- **Output:** `Uint8Array` (Mảng byte).
- **Thuật toán/Các bước thực hiện:**
 - Decode chuỗi Base64 thành chuỗi nhị phân thông qua hàm `atob`.
 - Khởi tạo một mảng `Uint8Array` với kích thước bằng chiều dài chuỗi.
 - Lặp qua từng ký tự, lấy mã vị trí ký tự (character code) gán vào mảng byte tương ứng và trả về.

2.2. Server-side

a. Utils functions

1. Hàm `isValidRoomId`

- **Mô tả:** Kiểm tra xem ID của phòng (channelId) có đúng định dạng UUIDv4 hợp lệ hay không.
- **Input:** `id`.
- **Output:** `boolean` (true hoặc false).

2. Hàm `checkRate`

- **Mô tả:** Kiểm tra và cập nhật bộ đếm giới hạn tần suất (Rate Limiting) cho một socket cụ thể, việc giới hạn này hạn chế một socket (người dùng) tạo quá nhiều kênh chat trong một khoảng thời gian.

- **Input:**
 - `map` (Map<string, number[]>): Bộ nhớ lưu trữ mốc thời gian.
 - `socketId` (string): ID của socket.
 - `config` (RateLimitConfig): Cấu hình giới hạn (số lượt tối đa và khoảng thời gian).
- **Output:** `boolean` (true nếu được phép, false nếu vượt quá giới hạn).
- **Các bước thực hiện:**
 - Lọc bỏ các mốc thời gian đã cũ ngoài khoảng `windowMs`.
 - Nếu số lượng mốc thời gian còn lại $\geq$ `max`, trả về `false`.
 - Nếu chưa, thêm mốc thời gian hiện tại vào danh sách, lưu lại vào Map và trả về `true`.

3. Hàm `clearRates`

- **Mô tả:** Xóa lịch sử giới hạn tần suất của một socket.
- **Input:** `socketId` (string).
- **Output:** Không có.
- **Các bước thực hiện:** Xóa key `socketId` khỏi cả `joinTimestamps` và `msgTimestamps`.

4. Hàm `roomSize`

- **Mô tả:** Lấy số lượng client hiện đang có mặt trong một phòng.
- **Input:** `roomId` (string).
- **Output:** `number` (số lượng socket).
- **Các bước thực hiện:** Truy vấn thông qua `io.sockets.adapter.rooms` của Socket.io.

5. Hàm `getSocketRoom`

- **Mô tả:** Tìm ID phòng chat (channel) mà socket hiện tại đang tham gia.
- **Input:** `socket` (đối tượng Socket).
- **Output:** `string | undefined` (ID phòng hoặc undefined).
- **Các bước thực hiện:** Lặp qua danh sách `socket.rooms` và trả về phần tử đầu tiên khác với chính `socket.id` (vì mặc định mỗi socket luôn nằm trong một phòng mang tên ID của nó).

b. Socket events

1. Sự kiện `join`

- **Mô tả:** Client yêu cầu tham gia vào một kênh trò chuyện (channel/room).
- **Payload (Input):** `{ channelId, publicKey }` (JoinPayload).
- **Hành động / Phản hồi:**
 - **Xác thực:** Trả về lỗi `error` nếu `channelId` không hợp lệ hoặc thiếu `publicKey`.
 - **Rate Limit:** Trả về lỗi `error` (`JOIN_RATE_LIMITED`) nếu vượt quá 5 lần join / 1 phút.
 - **Sức chứa:** Trả về lỗi `error` (`ROOM_FULL`) nếu phòng đã có đủ 2 người.
 - **Chấp nhận:** Gọi `socket.join(channelId)`. Sau đó phát sự kiện `peer-key` kèm `publicKey` cho người dùng đã ở sẵn trong phòng (nếu có).

2. Sự kiện `peer-key`

- **Mô tả:** Client gửi lại khóa công khai của mình để đáp lại khi có người mới join.
- **Payload (Input):** `{ channelId, publicKey }` (PeerKeyPayload).
- **Hành động / Phản hồi:** Nếu payload hợp lệ và socket đang ở đúng phòng, server chuyển tiếp (relay) sự kiện `peer-key` đến người đối diện trong phòng.

3. Sự kiện `message`

- **Mô tả:** Client gửi một tin nhắn đã được mã hóa.
- **Payload (Input):** `EncryptedPayload` (Chứa `channelId`, bản mã, vector khởi tạo...).
- **Hành động / Phản hồi:**
 - Kiểm tra socket có đang ở đúng phòng hay không.
 - Kiểm tra Rate Limit. Nếu vượt quá 60 tin nhắn / 1 phút, trả về lỗi `error` (`FLOOD_LIMIT`).
 - Nếu hợp lệ, chuyển tiếp toàn bộ payload qua sự kiện `message` đến người đối diện.

4. Sự kiện `delivered`

- **Mô tả:** Client báo cáo đã nhận được tin nhắn.
- **Payload (Input):** `{ id, channelId }`.
- **Hành động / Phản hồi:** Chuyển tiếp ID của tin nhắn qua sự kiện `delivered` cho người đối diện để họ cập nhật giao diện (hiển thị trạng thái "đã nhận").

5. Sự kiện `typing`

- **Mô tả:** Client báo cáo trạng thái đang gõ phím (payload được mã hóa).
- **Payload (Input):** `{ channelId, ciphertext, iv }` (TypingPayload).

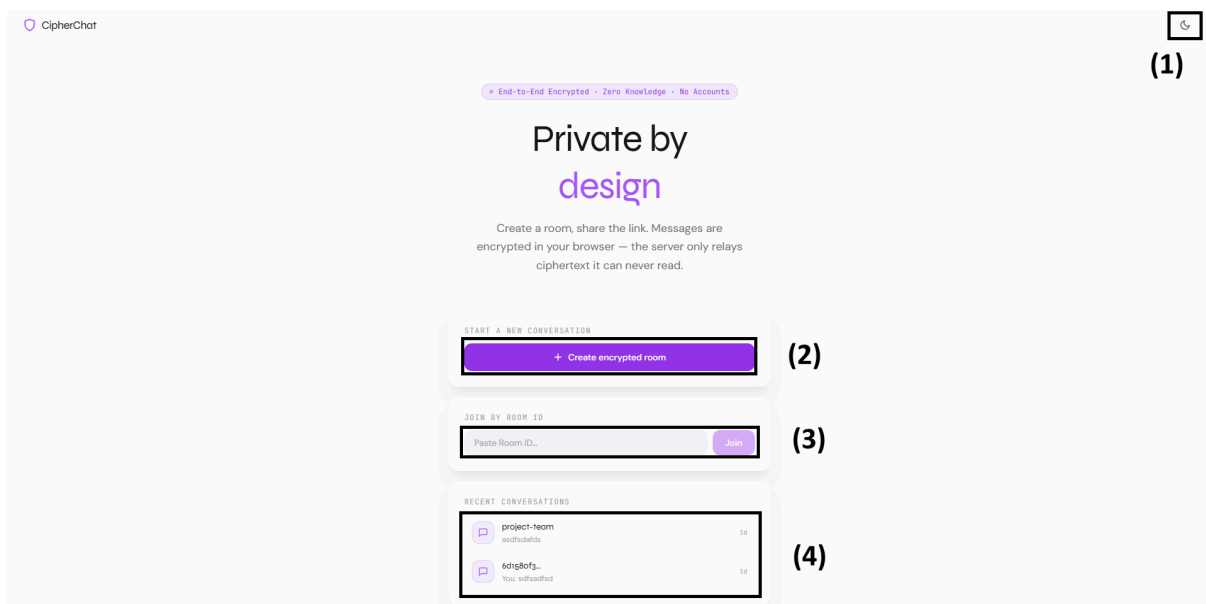
- **Hành động / Phản hồi:** Chuyển tiếp sự kiện `typing` cho người đối diện trong phòng.

6. Sự kiện `leave` / `disconnect` & adapter `leave-room`

- **Mô tả:** Xử lý việc client chủ động rời phòng hoặc mất kết nối mạng.
- **Payload (Input):** `{ channelId }` (đối với `leave`).
- **Hành động / Phản hồi:**
 - Gọi hàm `clearRates` để xóa bộ đếm giới hạn của socket.
 - **Adapter `leave-room`:** Đây là cách xử lý thông minh để bắt mọi trường hợp thoát phòng (chủ động hoặc rớt mạng). Bỏ qua trường hợp phòng mặc định (trùng ID). Khi xảy ra thoát phòng, tự động phát sự kiện `peer-left` cho người ở lại biết đối tác đã rời đi.

3. Screens

3.1. Home Page (`/`)

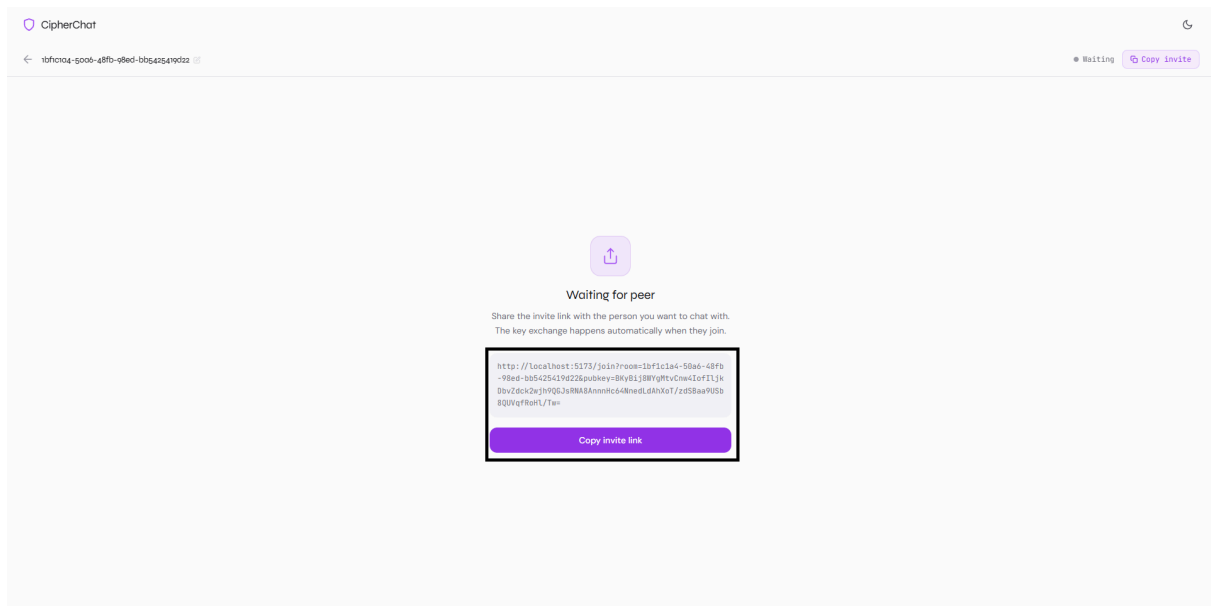


Các chức năng chính của Home Page:

1. Đổi theme
2. Tạo một cuộc trò chuyện mới
3. Tham gia vào một cuộc trò chuyện bằng ID
4. Danh sách các cuộc trò chuyện đã tham gia được lấy từ IndexedDB

3.2. Chat Page (`/room/: channelId`)

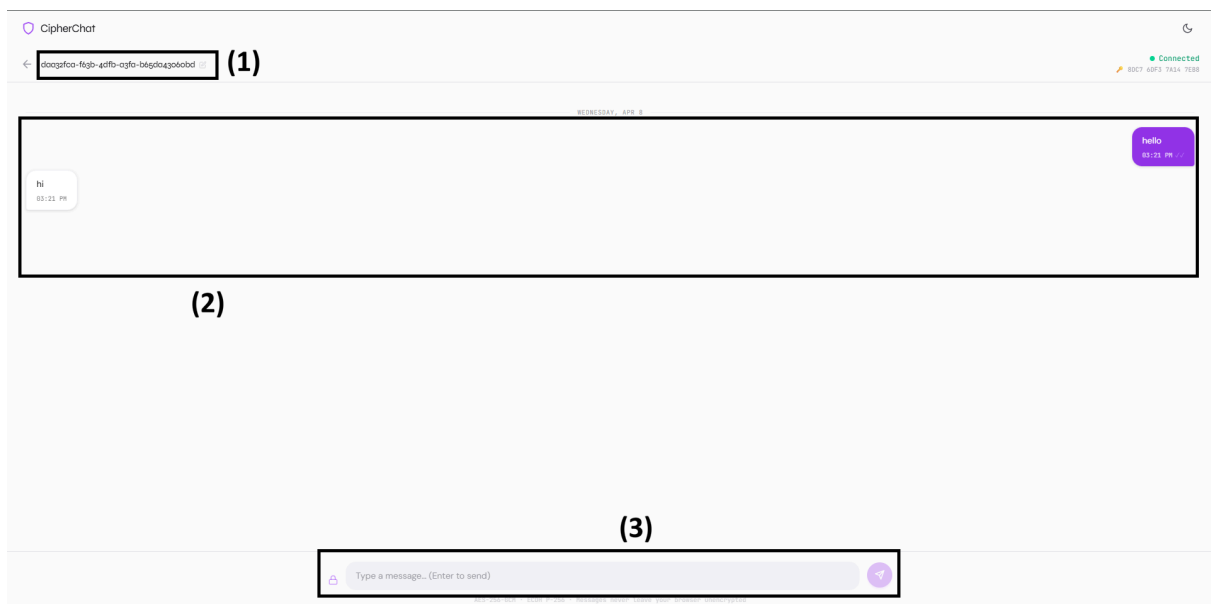
a. Giao diện chờ kết nối



Link gửi cho đối phương để tham gia phòng chat, gồm:

- ID của channel
- Public key của người tạo phòng: Để so sánh với public key khi trao đổi sau khi cả 2 cùng tham gia phòng, đảm bảo người gửi link và người đang chat có cùng public key

b. Giao diện kết nối thành công và bắt đầu nhắn tin



Các chức năng chính khi chat:

1. Đổi tên/nickname của channel (mặc định là ID rất khó nhớ)

2. Danh sách tin nhắn đã chat
3. Ô nhập tin nhắn